



In this assignment, you will design caches -cache controllers- in both gem5 simulator and Verilog HDL.

[70 Points] Set Associative Cache and Benchmarking in gem5 Simulator

gem5 (<https://www.gem5.org>) is an architectural simulator library written in C++ and Python that lets you simulate many instruction set architecture (x86, ARM, MIPS, RISC-V, etc.) based processors. In this part, you will add a configurable set associative cache from scratch and run and simulate CoreMark benchmark program (<https://github.com/eembc/coremark>) on x86 architecture, comparing it across different cache configurations.

Cache requirements are as follows:

- It will be **set associative**, and if the number of ways is specified as 1 then it will become a **direct mapped cache**.
- It will be used as Level 1 Instruction and Data caches.
- It will have these parameters:
 - **cache_size (Param.MemorySize)**: The total size of the cache. Default value: 1kB
 - **cache_way (Param.Int)**: Number of ways of the set associative cache. A value of 1 makes it a direct-mapped cache. Default value: 2
 - **cache_latency (Param.Cycles)**: Number of cycles taken on a hit to response. Default value: 1
 - **cache_replacement_policy (Param.String)**: The replacement policy to be used. The module must support the following two policies (Default value: LRU):
 - **LRU** (Least Recently Used)
 - **MRU** (Most Recently Used)
 - **cache_write_policy (Param.String)**: The write policy to be used. The module must support the following two policies (Default value: WT):
 - **WT** (Write-Through)
 - **WB** (Write-Back)
- The simulation object must keep these statistics:
 - **numberOfHits**: Number of successful requests
 - **numberOfMisses**: Number of failed requests
 - **totalRequests**: Number of total requests
 - **hitPercentage**: $100 \times \text{numberOfHits} / \text{totalRequests}$
 - **numberOfEvictions**: Number of cache blocks evicted from the cache.



Before adding a cache to gem5, you need to install necessary dependencies and build the library correctly.

Note: If you are a Windows user, use an x86 Ubuntu virtual machine on Ubuntu WSL (<https://ubuntu.com/wsl>) or VirtualBox (<https://www.virtualbox.org>) and follow the Ubuntu steps below. If you are on macOS, use VirtualBox or Docker (<https://www.docker.com>) with an x86 Ubuntu VM and follow the instructions for Ubuntu.

Install the tools required to build the library as follows:

```
sudo apt install build-essential
sudo apt install git
sudo apt install gcc g++
sudo apt install m4
sudo apt install scons
sudo apt install zlib1g
sudo apt install zlib1g-dev
sudo apt install python3-dev
```

Clone gem5 repository from GitHub and checkout commit `d22064c1c05f5857431219da888de7e29a268c16` (gem5 version 25.0.0.0). (You can also download it from <https://github.com/gem5/gem5/archive/refs/tags/v25.0.0.0.zip>):

```
git clone https://github.com/gem5/gem5
cd gem5
git checkout d22064c1c05f5857431219da888de7e29a268c16
python3 -m pip install -r requirements.txt
```

After obtaining the repository, build gem5 simulator for X86 instruction set architecture as follows:

```
cd gem5
scons build/X86/gem5.opt -j1
```

This will produce the gem5 simulator binary (`gem5.opt`) under `gem5/build/X86` directory. For faster build times, you can assign more threads in `-j1` flag instead of "1". You can find the maximum number of threads that you can assign via `nproc` command in the terminal.

After ensuring that the library has been successfully built, now you can add a custom cache to gem5 system. Create or modify following files in gem5 system to implement your custom cache inside `gem5/src/mem/cache` directory (you need to add **CustomCache** as a SimObject to the build system of it):

- customcache.hh
- customcache.cc
- CustomCache.py
- SConscript

After the modifications are completed, navigate to gem5 folder and rebuild the library:

```
cd gem5
scons build/X86/gem5.opt -j1
```

After ensuring that the library has been successfully rebuilt, you need to run a program in the simulator to measure the cache performance. Clone *CoreMark* benchmark (You can also download it from <https://github.com/eembc/coremark/archive/refs/tags/v1.01.zip>) and compile it for X86 Linux:



```
git clone https://github.com/eembc/coremark
cd coremark
make compile PORT_DIR=linux
```

If *CoreMark* has been successfully compiled, you will have **coremark.exe** binary inside of *coremark* directory.

Now, for running a system simulation in gem5 with your custom cache and running the coremark binary, you need to create a simulation script for that. Create a python script named **simulate.py** and build the gem5 system with **TimingSimpleCPU** and **CustomCache** objects. Use just L1 instruction and data caches. Let your memory be **DDR4_2400_8x8** and has **1GB** size. Let your maximum instructions to simulate **10000000** (10 million). You should be able to get parameters of custom cache via flags in this script as written below and simulate coremark with specific configurations like this:

```
cd gem5
build/X86/gem5.opt simulate.py \
    --program="./coremark.exe" \
    --outdir="./output" \
    --cache-size=1kB \
    --cache-way=2 \
    --cache-latency=1 \
    --cache-replacement-policy="LRU" \
    --cache-write-policy="WT"
```

Simulate coremark using all different replacement and write policy configurations and write a simple report as **report.txt** by itemizing. After you simulate the program, you will get statistics inside **output** directory. Mention about these statistics and how it behaves for different configurations, which config you got the best results for coremark, how each parameter affects the cache performance. Explain if there is a significant difference between set associative cache and direct mapped cache and why. Answer if better cache configurations and even 0 miss is possible. Try adding other objects (but submit pure version of the sim code) and answer if adding other objects such as a branch predictor affects cache performance or not and how.

You can use these additional resources for adding a cache to gem5:

Learning gem5 Tutorial

https://www.gem5.org/documentation/learning_gem5/introduction

Learning gem5 Tutorial: Creating a simple configuration script

https://www.gem5.org/documentation/learning_gem5/part1/simple_config

Learning gem5 Tutorial: Creating a very simple SimObject

https://www.gem5.org/documentation/learning_gem5/part2/helloobject

Learning gem5 Tutorial: Creating a simple cache object

https://www.gem5.org/documentation/learning_gem5/part2/simplecache

gem5 Bootcamp Videos

https://www.youtube.com/watch?v=6ThcR6jKwkc&list=PL_hVbFs_loVR_8ntTTmmG6YEq3Po_4snu&index=1



[40 Points] Fully Associative Cache in Verilog

In this part, you need to implement a custom data cache controller in Verilog HDL. You should be able to read and write data to the cache at each clock tick, assuming that the data is constantly coming from main memory. Write the logic inside `dcache_controller.v` attached by ensuring these requirements:

- It will be used as a data cache in the processor that is using **8-byte (double-word) addressing**.
- It will be **fully associative** and have **2KB** size.
- The address and data widths are **64 bits**.
- The cache write policy is **write-through**.
- The cache replacement policy is **FIFO (first-in-first-out)**.
- All reads and writes are aligned and are taking 1 cycle to operate.

***Note:** 2KB size does not contain tag and valid bits.*

***Note:** You may assume any features not specified that you think are necessary after meeting these requirements.*

Submission (Deadline: 07.12.2025 23.59)

⚠ You can use online resources or generative AI tools however we will do a plagiarism check across all codes and any matches will get 0 points from the homework. If you do it yourself -even with some help of AI-, you have nothing to worry about.

- 1-) customcache.hh
- 2-) customcache.cc
- 3-) CustomCache.py
- 4-) SConscript
- 5-) simulate.py
- 6-) report.txt
- 7-) dcache_controller.v

Compress these files and save it as <Surname_StudentNumber>.zip and upload it to <https://uzak.etu.edu.tr>.